PyTorch

- What is PyTorch?
- PyTorch vs TensorFlow
- Overview of PyTorch Ecosystem
    - Torch, torchvision, torchaudio, torchtext
- Installing PyTorch
- TensorFlow vs PyTorch for Neural Networks
- Introduction to Jupyter Notebooks and Google Colab for PyTorch

---

2. PyTorch Basics

2.1 Tensors

- What is a Tensor?
- Creating Tensors in PyTorch
    - `torch.Tensor()`, `torch.zeros()`, `torch.ones()`, `torch.rand()`
    - Tensor from NumPy arrays
    - Random and Identity Tensors
- Tensor Operations:
    - Arithmetic Operations (Addition, Subtraction, etc.)
    - Indexing, Slicing, and Reshaping
    - Broadcasting in PyTorch
    - Conversions: Tensor to NumPy and Back

2.2 Autograd and Backpropagation

- Introduction to Autograd
    - Gradient Calculation with `torch.autograd`
    - Tensors with `requires_grad=True`
- Backpropagation in Neural Networks
    - Computing Gradients
    - Chain Rule and Automatic Differentiation
    - Manual Gradient Updates
- Using `torch.no_grad()` for Inference

2.3 GPU Support

- Setting Up PyTorch with CUDA
- Moving Tensors to GPU: `tensor.cuda()` or `tensor.to(device)`

- Leveraging Multiple GPUs (`torch.nn.DataParallel`)
- Performance Considerations for GPU Usage

---

3. PyTorch Neural Networks

3.1 Building Neural Networks

- Introduction to `torch.nn.Module`
- Defining Layers and Forward Pass:
    - Linear Layers (`torch.nn.Linear`)
    - Activation Functions (ReLU, Sigmoid, Tanh, etc.)
    - Loss Functions (MSE, Cross-Entropy, etc.)
- Creating a Simple Feedforward Network
- Customizing Forward Method

3.2 Training Neural Networks

- Defining Loss Functions (`torch.nn.CrossEntropyLoss`, `torch.nn.MSELoss`)
- Optimizers (`torch.optim.SGD`, `torch.optim.Adam`)
- Forward and Backward Propagation in Training Loop
- Training a Model on Training Data
- Evaluation (Train/Test Split)

3.3 Overfitting and Regularization

- Techniques to Avoid Overfitting
    - Dropout (`torch.nn.Dropout`)
    - L2 Regularization (Weight Decay)
    - Early Stopping
- Cross-Validation
- Hyperparameter Tuning

---

4. Advanced PyTorch Architectures

4.1 Convolutional Neural Networks (CNNs)

- Understanding CNN Architecture:
    - Convolutional Layers (`torch.nn.Conv2d`)

    - Pooling Layers (`torch.nn.MaxPool2d`, `torch.nn.AvgPool2d`)
    - Flattening and Fully Connected Layers
- Building a CNN Model in PyTorch
- Visualizing Feature Maps
- CNN Applications:
    - Image Classification (CIFAR-10, MNIST)
    - Object Detection (YOLO, Faster R-CNN)

4.2 Recurrent Neural Networks (RNNs)

- Understanding RNN Basics
    - Simple RNN Layers (`torch.nn.RNN`)
    - LSTM Layers (`torch.nn.LSTM`)
    - GRU Layers (`torch.nn.GRU`)
- Time-Series Forecasting
- Natural Language Processing (NLP) with RNNs
- Sequence to Sequence Models for Machine Translation

4.3 Transformers

- Understanding Attention Mechanism
- Building Transformer Models in PyTorch
- Using Pretrained Models (BERT, GPT, T5) with Hugging Face
- Applications in NLP: Text Summarization, Translation, Text Generation

4.4 Generative Models

- Variational Autoencoders (VAEs)
    - Encoder and Decoder Networks
    - Latent Variable Models
- Generative Adversarial Networks (GANs)
    - Architecture of GANs (Generator vs Discriminator)
    - DCGAN, CycleGAN, StyleGAN
    - Applications: Image Synthesis, Style Transfer

---

5. Model Training, Evaluation, and Hyperparameter Tuning

5.1 Training Loop

- Implementing Custom Training Loops

- Tracking Losses and Metrics
- Using `torch.utils.data.DataLoader` for Batch Processing

5.2 Hyperparameter Tuning

- Grid Search and Random Search
- Hyperparameter Optimization with Libraries like Optuna, Ray Tune, or Hyperopt

5.3 Model Evaluation

- Evaluating Performance on Test Dataset
- Precision, Recall, F1 Score, AUC
- Visualizing Confusion Matrix, ROC Curves

---

6. Data Handling and Augmentation

- Dataset Class and DataLoader (`torch.utils.data.Dataset`, `DataLoader`)
- Custom Dataset for Image, Text, and Tabular Data
- Data Augmentation for Images
    - Using `torchvision.transforms`
    - Random Cropping, Horizontal Flip, Normalization
- Text Data Handling
    - Tokenization and Padding
    - Working with Embedding Layers (`torch.nn.Embedding`)

---

7. Transfer Learning in PyTorch

- Pretrained Models from `torchvision.models`
    - VGG, ResNet, AlexNet, DenseNet
    - Fine-tuning Pretrained Models
- Feature Extraction vs Fine-Tuning

---

8. Model Deployment

8.1 Saving and Loading Models

- Saving and Loading Model Weights (`torch.save`, `torch.load`)
- Serializing Models (`torch.jit` for TorchScript)
- Exporting Models to ONNX

8.2 Model Inference

- Performing Inference (Without Gradients)
- Optimizing Inference with TorchScript and JIT

8.3 Deployment

- Serving PyTorch Models with FastAPI or Flask
- Using `torchserve` for Model Serving
- Deploying Models on Cloud Platforms (AWS, GCP, Azure)

---

9. Advanced PyTorch Features

- Custom Loss Functions and Layers
- Understanding `torch.nn.functional` for Advanced Operations
- Using Mixed Precision Training for Faster Models
- Distributed Training with `torch.nn.DataParallel` and `torch.nn.parallel.DistributedDataParallel`

---

10. PyTorch Ecosystem

- Using `torchvision` for Image-based Projects
    - Pretrained Models, Image Transformations, and Datasets
- Using `torchtext` for NLP Tasks
    - Tokenization, Vocabularies, and Embeddings
- Using `torchaudio` for Audio Processing
- Using `torchmetrics` for Metric Calculations

---

11. Real-World Applications and Case Studies

Beginner Projects

- Handwritten Digit Classification (MNIST) with MLP
- CIFAR-10 Image Classification with CNN

Intermediate Projects

- Text Classification with RNNs (IMDB Dataset)
- Time-Series Prediction with LSTMs

Advanced Projects

- Object Detection with Faster R-CNN
- Building a GAN for Image Generation
- Natural Language Processing with Transformers (BERT or GPT)

---

12. Trends and Research in PyTorch

- Latest Developments in PyTorch
- PyTorch 2.0 Features (if applicable)
- Research Papers and Implementations
- Participating in Kaggle Competitions with PyTorch